FLOWOS

SOLUTION ARCHITECTURE

Multi-Tenant WhatsApp AI Sales Platform for SMEs

Project	FlowOS — AI Sales Agent Platform
Document Type	Solution Architecture & Proof of Concept Design
Industry	SME Automation / Conversational Commerce
Target Market	Nigeria
Version	v1.0 — POC Phase
Date	May 2026

s

1. Executive Summary

FlowOS is a multi-tenant SaaS platform that equips small and medium-sized businesses with an AI-powered WhatsApp sales agent. Each business that signs up on the platform receives its own isolated environment: a dedicated WhatsApp Business number, a self-service dashboard to configure their product catalogue and knowledge base, and an AI agent that handles customer conversations 24/7.

The AI does not simply answer questions. It detects purchase intent, qualifies leads, collects booking details, and escalates complex interactions to a human agent with full conversation context intact. Business owners monitor all activity from a real-time dashboard and are alerted instantly when human attention is required.

A super-admin panel gives the platform operator visibility across all tenants — total businesses, aggregated conversations, and platform-level health metrics.

Core Objectives

- Deploy a WhatsApp-first AI sales agent for each SME tenant with zero code required from the business owner.
- Capture and qualify leads (cold / warm / hot) automatically from customer conversations.
- Enable conversational booking and inquiry flows without human intervention.
- Provide business owners with a real-time dashboard for conversations, leads, bookings, and analytics.
- Route complex conversations to human agents with full AI-generated context summaries.
- Serve multiple independent SME tenants from a single backend with complete data isolation.
- Keep infrastructure costs below $130/month at POC stage, scaling cost linearly with tenant count.

2. System Overview

The platform has three primary actors and two operational layers.

Actors

- End Customer: A person messaging the business on WhatsApp. They never interact with the platform directly — only through the WhatsApp chat interface.
- Business Owner / Staff: An SME that signs up on FlowOS. They access the web dashboard to manage their knowledge base, view leads and bookings, and respond to escalated conversations.
- Super Admin (Platform Operator): The FlowOS team. They access the admin panel to manage all tenants, view platform metrics, and activate or deactivate accounts.

Operational Layers

- Conversation Layer (WhatsApp): Where customers interact. One WhatsApp Business number per SME tenant. Each number is connected to the FlowOS backend via the official WhatsApp Business API (WABA).
- Management Layer (Web Dashboard): Where business owners work. A React web application that each SME logs into to manage their bot, view customer activity, and handle escalations.

Tenant Isolation Model

Every database record carries a business_id foreign key. Supabase Row-Level Security (RLS) policies enforce that queries can only return rows matching the authenticated tenant's ID. This means even if application code has a bug, the database itself prevents cross-tenant data access. This is the most reliable multi-tenancy isolation pattern available without a separate database per tenant.

3. Optimal Technology Stack

The stack below was selected for three criteria: lowest infrastructure cost at POC scale, fastest time to ship, and cleanest path to production scale. Every tool chosen has a free or very low-cost tier that is sufficient for a POC with under 20 active tenants.

Layer	Chosen Tool	Alternatives Considered	Rationale
WhatsApp API	360dialog (Official BSP)	Twilio, WATI	€49/mo flat, direct Meta access, multi-WABA support, developer-first docs
Backend Runtime	Node.js (Express/Fastify)	Python/FastAPI	Ideal for real-time webhook handling; async I/O for concurrent tenants
Database	PostgreSQL on Supabase	MongoDB, MySQL	Row-Level Security (RLS) natively enforces tenant isolation; real-time built-in
AI / LLM	Claude Sonnet (Anthropic) or GPT-4o-mini	Self-hosted Llama	Pay-per-use; no GPU infra; Claude excels at instruction-following for sales flows
Vector Search	pgvector (PostgreSQL extension)	Pinecone, Weaviate	Eliminates a separate vector DB cost; cosine similarity at moderate scale is fine in Postgres
Frontend Dashboard	React + Tailwind (Vercel)	Vue, Angular	Fastest iteration; free hosting on Vercel; component ecosystem
Backend Hosting	Render (Starter/Standard)	Railway, Fly.io, AWS	Predictable pricing, no cold starts on paid plans, managed deploys
Auth	Supabase Auth (JWT)	Auth0, Firebase Auth	Free tier; integrates with RLS policies; handles multi-role (owner / staff / admin)
Notifications	WhatsApp API + Resend (email)	SendGrid, Twilio SMS	WhatsApp is free in 24hr window; Resend free tier covers 3K emails/mo
Queue / Background Jobs	BullMQ + Redis (Upstash)	RabbitMQ, SQS	Upstash Redis is serverless/free tier; BullMQ handles async message processing

Why pgvector Instead of Pinecone

Pinecone starts at $70/month for a managed vector database. pgvector is a PostgreSQL extension that runs inside the same Supabase database you are already paying for. At POC scale — with each SME having fewer than 500 products and 200 FAQ entries — pgvector cosine similarity search returns results in under 50ms. The switch to a dedicated vector database is a production optimisation, not a POC requirement.

Why 360dialog Instead of Twilio for WhatsApp

360dialog charges a flat $59/month per WhatsApp Business Account (WABA) with direct Meta API access and zero message markup. Twilio charges a platform fee on every message on top of Meta's rates, which becomes expensive fast with high-volume tenants. For a multi-tenant product where each SME brings their own WABA,

360dialog's Partner API is the correct choice: it allows you to manage unlimited WABA accounts through one API key.

Why Supabase Instead of Raw PostgreSQL

Supabase provides managed PostgreSQL with Row-Level Security, built-in authentication with JWT support, a real-time subscription layer (for live dashboard updates), and the pgvector extension already enabled. The free tier supports up to 500MB data and 2GB file storage — sufficient for the POC. Migrating to a dedicated PostgreSQL instance later requires changing only the connection string.

4. Data Architecture

4.1 Database Schema

All tables live in a single PostgreSQL database. Every table that contains tenant-specific data includes a business_id column. Supabase RLS policies automatically filter queries by this ID, enforcing isolation at the database engine level.

Table	Key Columns	Purpose
businesses	id, name, whatsapp_number, waba_id, api_key, plan, is_active, created_at	One row per SME tenant
business_users	id, business_id, name, email, role (owner/staff), password_hash, created_at	Staff who log into the dashboard
customers	id, business_id, whatsapp_number, name, email, created_at	End-customers per tenant
conversations	id, business_id, customer_id, status (open/resolved/escalated), ai_enabled, created_at	Chat session per customer per business
messages	id, conversation_id, direction (in/out), content, intent, confidence_score, created_at	Every individual WhatsApp message
leads	id, business_id, customer_id, conversation_id, score (cold/warm/hot), details_json, created_at	Captured lead per sales conversation
bookings	id, business_id, customer_id, service, date_time, status, notes, created_at	Appointment records
knowledge_base	id, business_id, type (product/faq/policy), title, content, embedding vector(1536), created_at	Business-specific KB entries with pgvector embeddings
products	id, business_id, name, description, price, category, is_available, embedding vector(1536)	Product catalogue per tenant
notifications	id, business_id, type, payload_json, sent_at, channel (whatsapp/email)	Alert records sent to business owners

4.2 Knowledge Base Design

Each business has two categories of knowledge base entries:

- Products: Name, price, category, description. The AI auto-generates a rich description from the product name and any details provided. The description is then embedded using the LLM embedding API and stored in the embedding column as a vector(1536). At query time, the customer's message is embedded and a cosine similarity search returns the top 5 most relevant products.
- FAQs / Support: Business policies, opening hours, delivery info, refund policy. These are chunked, embedded, and stored separately. When a customer asks a support question, the system retrieves the most relevant FAQ chunks and injects them into the LLM prompt.

4.3 Conversation Memory

The last N messages for each conversation are fetched from the messages table and appended to every LLM prompt as conversation history. For the POC, N = 20 messages (approximately the last 10 exchanges). This provides context continuity without excessive token costs. Long-term memory (cross-session) is handled by the customer record and lead record, which store extracted entities from past conversations.

5. AI Agent Architecture

5.1 Intent Classification

Every inbound message is classified into one of five intent categories before a response is generated. This classification drives the routing logic:

Intent	Trigger Example	System Action
INQUIRY	'What services do you offer?' / 'How much is the X?'	Retrieve from KB using pgvector search; respond with accurate data
BOOKING	'I want to book an appointment for Friday'	Initiate structured booking collection flow (service, date, time, name)
LEAD	'I'm interested but need to think about it'	Capture contact details; tag as warm lead; notify business owner
PURCHASE	'I want to order 2 of the X'	Collect order details; confirm to customer; create lead/booking record
ESCALATE	'I want to speak to a human' / low confidence score	Pause AI; send conversation + summary to staff via dashboard and WhatsApp

5.2 Prompt Architecture

Each LLM call is assembled from four components:

1. System Prompt: Defines the AI's persona (sales agent for [Business Name]), tone, and strict rules (never invent prices; only use provided knowledge base; escalate if unsure).
2. Business Context: Injected dynamically per tenant — business name, operating hours, current date, and any active promotions.
3. Retrieved Knowledge: Top-5 cosine similarity results from pgvector for the customer's query. Products and FAQs are retrieved separately and combined.
4. Conversation History: Last 20 messages from the current session, formatted as a dialogue.

5.3 Confidence Score & Escalation

The LLM is prompted to return a structured JSON response containing the reply text and a confidence score between 0 and 1. The backend checks this score against a configurable threshold (default: 0.6). If the score falls below the threshold, or if the customer explicitly requests a human, the following escalation sequence runs:

5. AI sends a message: 'Let me connect you with our team who can help you better.'
6. Conversation status is set to ESCALATED in the database.
7. AI is disabled for this conversation session.
8. Backend generates a summary using the LLM: customer name, issue type, key entities mentioned, sentiment.
9. Summary + full chat link is sent to the business owner via WhatsApp notification and dashboard alert.
10. Assigned staff member takes over the conversation from the dashboard; messages are sent via the same WhatsApp API.

Appendix: Glossary

Term	Definition
WABA	WhatsApp Business Account — the Meta-verified account associated with a business WhatsApp number. Each SME tenant on FlowOS has one.
BSP	Business Solution Provider — a Meta-approved partner (such as 360dialog) that provides access to the WhatsApp Business API and manages billing.
pgvector	A PostgreSQL extension that enables storing and querying high-dimensional vector embeddings directly in the database, enabling semantic similarity search.
Embedding	A numerical vector representation of text, generated by an AI model, that captures semantic meaning and allows similarity comparisons between texts.
Cosine Similarity	A mathematical measure of how similar two vectors are, used to find the most relevant knowledge base entries for a customer's query.
RLS	Row-Level Security — a PostgreSQL feature that enforces access rules at the database level, ensuring tenants can only query their own rows.
Multi-tenancy	An architecture where a single platform instance serves multiple customers (tenants), with each tenant's data completely isolated from others.
RAG	Retrieval-Augmented Generation — an AI pattern where relevant documents are retrieved from a knowledge base and injected into the LLM prompt to ground responses in factual data.
BullMQ	A Node.js queue library built on Redis, used for processing background jobs (such as sending notifications) asynchronously without blocking the main request cycle.
JWT	JSON Web Token — a signed token issued on login, sent with every API request to prove identity and role without hitting the database on each call.